

Arithmetic: Our First Interpreter

CS 350

Dr. Joseph Eremondi

February 21, 2025

Interpreters: Overview

Extending Curly-Arith to Curly-Cond

Interpreters: Overview

What is an interpreter?

- A program

What is an interpreter?

- A program
 - Written in some language A

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program
- Different from compiler

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program
- Different from compiler
 - Compiler is only a translator

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program
- Different from compiler
 - Compiler is only a translator
 - Usually to machine language

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program
- Different from compiler
 - Compiler is only a translator
 - Usually to machine language
 - Leaves the running of the code to CPU or another interpreter

What is an interpreter?

- A program
 - Written in some language A
 - Takes as input a program written in language B
 - Executes that program
- Different from compiler
 - Compiler is only a translator
 - Usually to machine language
 - Leaves the running of the code to CPU or another interpreter
 - E.g. Typescript compiler emits Javascript, leaves running of code to JavaScript run-time

The Church Turing Thesis



Alonzo
Church

*(lambda
calculus)*



Kurt Gödel

*(general
recursive
functions)*



Alan Turing

*(Turing
machines)*

Turing Completeness

Turing Completeness

- The following can all simulate each other:

Turing Completeness

- The following can all simulate each other:
 - Turing Machines

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)

The Church Turing Thesis

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*

The Church Turing Thesis

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*
 - Any language with `while` loops or recursion is Turing Complete

The Church Turing Thesis

Turing Completeness

- The following can all simulate each other:
 - Turing Machines
 - General-recursive functions
 - Lambda calculus (we'll see later)
- We call a programming language that can simulate a Turing machine *Turing Complete*
 - Any language with `while` loops or recursion is Turing Complete

All Turing Complete Languages can simulate each other

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language
- The features of a language you're interpreting are *completely unrelated* to the features of the language the interpreter is written in

Turing Completeness and Interpreters

- You can write an interpreter for any language in any Turing-complete language
- The features of a language you're interpreting are *completely unrelated* to the features of the language the interpreter is written in
 - Sometimes you can piggyback on the implementation language features, but that's a matter what's *convenient*, not what's *possible*

Keeping it all straight

- The implementation language is NOT the language you're interpreting

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets
 - Textbook calls them SMoL (Standard Model of Languages)

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets
 - Textbook calls them SMoL (Standard Model of Languages)
 - Write Curly programs in Racket files using quotation

Keeping it all straight

- The implementation language is NOT the language you're interpreting
- In this class, the implementation language is Racket/Flit
- We'll write interpreters for a bunch of small languages
 - We'll call them "Curly" because we write them with curly brackets
 - Textbook calls them SMoL (Standard Model of Languages)
 - Write Curly programs in Racket files using quotation
 - Backtick turns them into S-Expressions for easy parsing

General Form of an Interpreter

General Form of an Interpreter

```
(define (interp [e : Expr]
               [x : SomeContext]
               ...
               [y : OtherContext])
  : Value
....)
```

- Expr is the is the AST datatype for whatever language we're interpreting

General Form of an Interpreter

```
(define (interp [e : Expr]
                [x : SomeContext]
                ...
                [y : OtherContext])
  : Value
....)
```

- Expr is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language

General Form of an Interpreter

```
(define (interp [e : Expr]
                [x : SomeContext]
                ...
                [y : OtherContext])
  : Value
....)
```

- Expr is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language
 - Initially we have no context arguments, and Value is very simple

General Form of an Interpreter

```
(define (interp [e : Expr]
                [x : SomeContext]
                ...
                [y : OtherContext])
  : Value
....)
```

- Expr is the AST datatype for whatever language we're interpreting
- What the context arguments and Value datatype are depend on the language
 - Initially we have no context arguments, and Value is very simple
 - Will get more complicated as we go through the course

Our First AST Type

- Exp is:

Our First AST Type

- Exp is:
 - Output of parsing

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate
- Value is the result of running a program

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`
 - For now, just numbers

Our First AST Type

- `Exp` is:
 - Output of parsing
 - Input to `interp`
 - Tree representation of the term to evaluate
- `Value` is the result of running a program
 - Output of `interp`
 - For now, just numbers

Our First AST Type

- Exp is:
 - Output of parsing
 - Input to interp
 - Tree representation of the term to evaluate
- Value is the result of running a program
 - Output of interp
 - For now, just numbers

```
(define-type-alias Value Number)
(define-type Exp
  (num [n : Number])
  (plus [left : Exp]
        [right : Exp])
  (times [left : Exp]
         [right : Exp]))
```

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations
 - Interpret sub-expressions recursively

Structure of Interpreter

- Idea: if we know what the values of sub-expressions are, we can then look at what kind of AST node we have to determine what to do with those values
 - How to get values from sub-expressions?
 - Recursion!
- `interp` is a recursive function on structure of syntax
 - Base cases are literals, translate directly into values
 - Recursive cases are operations
 - Interpret sub-expressions recursively
 - Combine according to value version of the operation

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

```
(define (interp [e : Exp] )
  : Value
  (type-case Exp e
    [(num n) n]
    [(plus l r)
     (+ (interp l) (interp r))]
    [(times l r)
     (* (interp l) (interp r))])
(define (run s-exp) (interp (parse s-exp)))
(run `3)
(run `{+ 2 5})
(run `{+ {+ 1 {+ 2 1}} {+ {+ 3 4} {+ 0 1000000}}})
```

Our First Interpreter

- Interpreting arithmetic, so values are just Flit Number
 - See Curly-Arith on URCourses

```
(define (interp [e : Exp] )
  : Value
  (type-case Exp e
    [(num n) n]
    [(plus l r)
     (+ (interp l) (interp r))]
    [(times l r)
     (* (interp l) (interp r))])
(define (run s-exp) (interp (parse s-exp)))
(run `3)
(run `{+ 2 5})
(run `{+ {+ 1 {+ 2 1}} {+ {+ 3 4} {+ 0 1000000}}}))
```

```
3
7
15
```

Extending Curly-Arith to Curly-Cond

Adding features

- Need to update

Adding features

- Need to update
 - AST definition

Adding features

- Need to update
 - AST definition
 - Parser

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if0 cond x y}`

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if  $\theta$  cond x y}`
 - Evaluates to x if cond evaluates to θ

Adding features

- Need to update
 - AST definition
 - Parser
 - Interpreter
- Example: `{if  $\emptyset$  cond x y}`
 - Evaluates to x if cond evaluates to $\emptyset$
 - Evaluates to y if cond evaluates to anything else

Updating the datatype

Updating the datatype

```
(define-type Expr
  (num [n : Number])
  (plus [left : Exp]
        [right : Exp])
  (times [left : Exp]
         [right : Exp]))
;;NEW
;; conditional expressions
(cnd [test : Exp]
     [thenBranch : Exp]
     [elseBranch : Exp])
```

Updating the parser

Updating the parser

```
[(s-exp-match? ~{ifo ANY ANY ANY} s)
 (cnd (parse (second (s-exp->list s)))
       (parse (third (s-exp->list s))
               (parse (fourth (s-exp->list s))))]
```

Updating the interpreter

Updating the interpreter

```
(define (interp [e : Exp] ) : Number
  (type-case Expr e
    [(num n) n]
    [(plus l r)
     (+ (interp l) (interp r))]
    [(times l r)
     (* (interp l) (interp r))]

    [(cnd test thenBranch elseBranch)
     (if (= 0 (interp test))
         (interp thenBranch)
         (interp elseBranch)
      )])
```

- If the test expression evaluates to 0, then evaluate the then-expression

Updating the interpreter

```
(define (interp [e : Exp] ) : Number
  (type-case Expr e
    [(num n) n]
    [(plus l r)
      (+ (interp l) (interp r))]
    [(times l r)
      (* (interp l) (interp r))]

    [(cnd test thenBranch elseBranch)
      (if (= 0 (interp test))
        (interp thenBranch)
        (interp elseBranch)
      )])
```

- If the test expression evaluates to 0, then evaluate the then-expression
 - Otherwise, evaluate the else-expression

- Why couldn't we just compare test to zero?

- Why couldn't we just compare test to zero?

Syntax vs. Values

- Why couldn't we just compare test to zero?

```
;; Interp case for If0  
;; bad version  
[(cnd test thenBranch elseBranch)  
 (if (equal? test (num 0))  
      (interp thenBranch)  
      (interp elseBranch))]
```

- What would {if0 {+ 0 0} 3 4} evaluate to?

Syntax vs. Values

- Why couldn't we just compare test to zero?

```
;; Interp case for If0  
;; bad version  
[(cnd test thenBranch elseBranch)  
 (if (equal? test (num 0))  
      (interp thenBranch)  
      (interp elseBranch))]
```

- What would {if0 {+ 0 0} 3 4} evaluate to?
 - Check would fail because it's not the *literal number* 0

Syntax vs. Values

- Why couldn't we just compare test to zero?

```
;; Interp case for If0  
;; bad version  
[(cnd test thenBranch elseBranch)  
 (if (equal? test (num 0))  
      (interp thenBranch)  
      (interp elseBranch))]
```

- What would {if0 {+ 0 0} 3 4} evaluate to?
 - Check would fail because it's not the *literal number 0*
 - Computer programs need to do computation

Keeping it all Straight

- We want equivalent expressions to produce equivalent results

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - if $\emptyset$ should work the same for $\emptyset$, $\{+ 1 -1\}$, $\{* \emptyset 5\}$ etc.

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - `if 0` should work the same for `0`, `{+ 1 -1}`, `{* 0 5}` etc.
- Always recursively interpret expression to a value before we try doing semantic operations

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - $if\ 0$ should work the same for 0 , $\{+ 1 -1\}$, $\{* 0 5\}$ etc.
- Always recursively interpret expression to a value before we try doing semantic operations
- When in doubt, follow the types

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - $if\ 0$ should work the same for 0 , $\{+ 1 -1\}$, $\{* 0 5\}$ etc.
- Always recursively interpret expression to a value before we try doing semantic operations
- When in doubt, follow the types
 - Exp is abstract syntax, uninterpreted expressions

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - `if 0` should work the same for `0`, `{+ 1 -1}`, `{* 0 5}` etc.
- Always recursively interpret expression to a value before we try doing semantic operations
- When in doubt, follow the types
 - `Exp` is abstract syntax, uninterpreted expressions
 - `Value` is the result of interpreting

Keeping it all Straight

- We want equivalent expressions to produce equivalent results
 - `if 0` should work the same for `0`, `{+ 1 -1}`, `{* 0 5}` etc.
- Always recursively interpret expression to a value before we try doing semantic operations
- When in doubt, follow the types
 - `Exp` is abstract syntax, uninterpreted expressions
 - `Value` is the result of interpreting
 - What we want to do addition/multiplication/zero-checking on